A Feature-Major Codebook for Memory-Efficient Sparse-Binary Self-Organizing Maps: Scaling a MEDLINE Atlas to 1.05 Million Neurons on a Single Consumer GPU

Andrew J. Amos

College of Medicine and Dentistry, James Cook University, Townsville, Queensland, Australia

Abstract

A self-organising map turns a large corpus into a browsable two-dimensional atlas, but building one at MEDLINE scale has been impractical: the best-matching-unit (BMU) search that dominates training is bound by the bandwidth needed to read the codebook every epoch. I show that this bottleneck is largely an artefact of codebook layout. Storing it feature-major with each feature's weights contiguous, W[v·M+i], recasts the search as a tiled sparse-dense product in which every loaded weight column is reused across a tile of samples. Varying only the layout, with implementation, precision and update rule held fixed, accelerates the BMU search by 4.5-8.5×. Because an exact-argmin BMU is invariant to how the codebook is stored, this gain costs nothing: held-out quantisation error agrees with a cuSPARSE baseline to within 0.5% at every map size. Against that baseline the advantage is a crossover rather than a constant: cuSPARSE.SOM is faster at small maps, SparseBin.SOM is 1.5× faster at 128×128 and 2.6× at 256×256, and at 512×512 it is the only one that runs at all on 24 GB. Paired with a radius-independent box-blur update and a convergence-based stopping rule, it trains a converged map over 29.9 million MEDLINE articles in about 72 s at 64×64 on one 24 GB GPU, and accommodates 262,144 neurons (512×512 edges) where every alternative algorithm I tested exceeds memory constraints. On a

Email address: andrew.amos@my.jcu.edu.au (Andrew J. Amos)
URL: https://orcid.org/0000-0002-9145-0212 (Andrew J. Amos)

141 GB H200 it reaches 1,048,576 neurons (1024×1024 edges) - to my knowledge the largest self-organising map yet reported. Held-out error follows a smooth power law with no elbow across three decades of map size, so the limit on resolution is compute rather than any breakpoint in the data. At matched work the design is ∼82× faster than MedSOM, the CUDA implementation behind our earlier MEDLINE atlases and, at 128×128, 621× faster than the best available multicore-CPU library.

Keywords: self-organizing maps, GPU computing, sparse matrix kernels, memory-bandwidth optimisation, CUDA, MEDLINE/MeSH

---

1. Introduction

The organisation of medical knowledge has practical consequences. Decisions about what to teach, what to research, and what to prioritise still rest largely on expert judgement which, however well-informed, carries the systematic biases of its authors - with documented costs to the diversity of the medical workforce and the completeness of medical curricula [1]. The peer-reviewed literature indexed in MEDLINE is the most comprehensive and least biased record of medical knowledge available, and techniques that summarise it can offer objective, structural evidence to complement expert judgement in curriculum development, for example by surfacing emerging topics for curriculum renewal [2]. Among such techniques the self-organising map (SOM) is distinctive in producing a discrete, addressable, two-dimensional atlas in which topical neighbourhoods are spatially contiguous and which can be read much as one reads a geographic map [3, 4]. Realising that promise across the whole corpus is, however, first of all a computational problem - and it is that problem this paper addresses.

A self-organizing map projects high-dimensional data onto a low-dimensional lattice of prototype vectors (called neurons in this manuscript) such that nearby neurons respond to similar inputs. The result is a discrete, addressable, and browsable atlas: every input is assigned to a grid cell, and the grid itself is laid out so that topical neighbourhoods are spatially contiguous. For a corpus on the scale of MEDLINE - tens of millions of articles indexed against a controlled vocabulary of tens of thousands of Medical Subject Headings (MeSH) [13] - such an atlas is an attractive organising structure, but producing it is computationally demanding.

The cost of SOM training is dominated by the best-matching-unit (BMU)

search: for each input, the algorithm must find the nearest of M prototype vectors (representing neurons). At corpus scale this search reads the codebook - the M×V table of prototype weights - repeatedly across many epochs. When the codebook is laid out node-major (each neuron's V weights stored contiguously), the per-warp access pattern is a strided gather across non-contiguous memory; at a codebook size far exceeding the L2 cache, every gather becomes an HBM round-trip and the kernel is bound by uncoalesced memory traffic. The BMU search is, in short, memory-bandwidth-bound, and the lever for accelerating it is the codebook layout.

In this paper I demonstrate that storing the codebook feature-major - the M weights for a given feature stored contiguously, W[v·M+i] - is the more efficient layout for sparse inputs. It makes each feature's weight column a coalesced read, and it enables a tiled BMU kernel in which a loaded weight column is reused across all samples in a tile that share that feature. The material difference is the access pattern rather than the values stored, so it does not depend on the inputs being binary; the same layout drives a sparse-float variant of the design (Table 8). Binary inputs are a specialisation on top of it, worth a further 2.1-2.8× speedup in per-epoch training time (5.5). I pair this layout with a factorised box-blur neighbourhood update whose cost is independent of the neighbourhood radius, and a convergence-based stopping rule, to obtain an end-to-end system that trains over a MEDLINE-scale corpus on a single commodity GPU.

A central observation organises the evaluation. An exact-argmin BMU returns the same assignment regardless of how the codebook is stored or which kernel computes it; the layout therefore affects only speed, never quality. This lets me make the efficiency claim rigorously: I demonstrate matched clustering quality (quantisation error, topographic error, dead-unit fraction) between SparseBin.SOM and a strong same-GPU library baseline, and attribute the entire speed-up to the BMU/layout axis. I deliberately restrict 'quality' here to intrinsic SOM quality; the scientific validity of the resulting MEDLINE atlas - whether it recovers the MeSH and citation-derived knowledge structure - is the subject of a companion paper [5]. The present method also scales our prior MEDLINE-SOM programme - a 350×350 cartographic atlas of medical knowledge [3], externally validated against an expert-derived textbook knowledge structure [4] - by more than an order of magnitude in neuron count.

Contributions. This paper makes the following contributions:

- A feature-major codebook layout for sparse SOMs, with the memory-access argument for why it coalesces reads and enables tile-level weight reuse (3.2).

- An SpMM-tile BMU kernel that reads only the distinct feature columns a tile touches, reuses each across the tile's samples, and fuses the argmin into the node sweep (3.3).

- Empirical confirmation that the BMU/layout axis affects only speed, isolating quality to the update rule, which makes the speed-up a verifiable free lunch (3.4, 5.3).

- An end-to-end system that trains a converged map over all 29.9 million MEDLINE articles in the 2026 baseline carrying at least five MeSH descriptors on one 24 GB GPU, remains the only implementation of those tested that runs at 262,144 neurons on that card (5.5), and on a 141 GB H200 reaches 1,048,576 neurons - the largest SOM I am aware of, exceeding the WEBSOM patent map on both corpus and map size (5.6).

- A scaling study showing that held-out quantisation error follows a smooth power law with no elbow across three decades of map size (5.6), and a matched-work efficiency comparison placing SparseBin.SOM $\sim$82$\times$ ahead of MedSOM and, at 128$\times$128, 621$\times$ ahead of the best multicore-CPU library, a ratio that climbs steeply with map size - end-to-end figures that bundle layout, update rule and precision, the isolated layout effect being the 4.5-8.5$\times$ of 5.1 (5.5).

The remainder of the paper reviews background and related work (2), develops the method (3), describes the experimental setup (4), presents results (5), and discusses implications, limitations, and reproducibility (6-9).

2. Background and related work

2.1. Self-organizing maps and quality measures

I use the batch SOM [10, 11]. Each training epoch proceeds in two phases: an assignment phase that maps every input to its BMU, and an update phase that replaces each neuron's prototype with the weighted mean of the inputs assigned to units in its neighbourhood, under a neighbourhood kernel that

shrinks over training. I report three standard quality measures [20]. Quantisation error (QE) is the mean distance between inputs and their BMU prototypes and measures how faithfully the codebook represents the data. Topographic error (TE) is the fraction of inputs whose first and second BMUs are non-adjacent on the lattice and measures how well the map preserves topology. The dead-unit fraction is the proportion of neurons that win no inputs. I compute QE under cosine distance for cross-implementation comparison and report the implementation's native Euclidean QE where relevant.

2.2. GPU and parallel SOM implementations

The package somoclu [23], the strongest openly available multicore-CPU baseline for this task, provides dense CPU, dense GPU, and sparse CPU kernels. Its GPU kernel is dense-only: at a vocabulary of ∼30,000 features a dense M×V codebook and dense input representation are infeasible (the codebook alone would run to petabytes across a map-size sweep), so the only somoclu kernel viable for MEDLINE is the multicore sparse-CPU kernel. Prior GPU SOMs, including our own earlier CUDA implementation (MedSOM; [3, 4]), accelerate the BMU search but use per-node update loops and node-major codebooks; I use MedSOM as a same-task GPU baseline in 5.5. A further distinction concerns how the best-matching unit is computed: most implementations - the classic Kohonen line, somoclu, and the cuSPARSE baseline, as well as the main model developed here, SparseBin.SOM - evaluate the exact Euclidean distance, carrying each node's $\|\mathrm{w}_i\|^2$ and leaving the codebook un-normalised, so their prototypes remain interpretable cell means and their quantisation error is directly comparable. MedSOM is the exception, L2-normalising its codebook once per epoch so the search collapses to a maximum dot product - faster, but it shifts the effective metric toward cosine and renders its quantisation error non-comparable; this is why, in 5, the exact-argmin implementations agree on quality and only MedSOM's QE is excluded from the comparison.

2.3. Sparse kernels, memory layout, and the roofline

The BMU search is a product between the sparse input matrix and the dense codebook, closely related to SpMM. Library SpMM (cuSPARSE) exposes a row- versus column-major output ordering (ORDER_ROW vs ORDER_COL) that corresponds directly to the feature-major versus node-major codebook distinction, letting me isolate the layout effect within a single library (5.2). Because the kernel is memory-bound, the relevant analytical

frame is the roofline model [22]: performance is set by achieved bandwidth and arithmetic intensity rather than peak FLOPs, which motivates a layout that maximises coalesced reads and data reuse.

2.4. How large can a SOM be, and how is its size chosen?

The largest SOM in the peer-reviewed literature is the WEBSOM culmination of Kohonen et al. (2000), which mapped 6,840,568 patent abstracts - represented as 500-dimensional random projections of weighted word histograms - onto a 1,002,240-node map. Two points are relevant here. First, the map size was not derived from any optimality metric: it was set by what was computationally feasible and by the browsing use-case, and the large grid was reached by an engineering shortcut - training a much smaller grid first, then inserting interstitial nodes and interpolating their weights. Quantisation error appears in that work as a quality justification, not as a size selector. Second, the only project operating at this scale therefore sized its map by budget and task, not by a breakpoint in the data.

The literature that does claim a useful size criterion operates orders of magnitude smaller. The most principled recent proposal, SMLSOM [14], selects the number of units automatically by deleting redundant ones under a minimum-description-length criterion, but is demonstrated on small real datasets. The growing/hierarchical family - the Growing Hierarchical SOM [19] and descendants such as RT-GSOM [18] - is evaluated on UCI-repository and small text collections (roughly 10^3-10^5 samples). Notably, these methods sidestep the flat-map size question rather than answer it: a GHSOM does not select a single optimal flat size but grows a hierarchy in which each region expands until its local quantisation error falls below a fraction of its parent's, governed by a local threshold rather than a global QE/TE elbow. In short, optimality-by-metric has been demonstrated only at benchmark scale, while the one $\sim 10^7$-sample SOM was sized by resources - context that frames the scaling result in 5.6.

More directly, the present work scales our own prior MEDLINE SOMs. A 350×350 (122,500-node) batch SOM of the MEDLINE corpus was used to build a cartographic atlas of medical knowledge [3], its map size selected from a topographic-error plateau. It was subsequently validated by projecting an expert-derived textbook knowledge structure - the reference lists of ten editions of a standard psychiatric textbook - onto the trained map [4]. That 350×350 map is the MedSOM baseline of 5.5; the feature-major method carries the same modelling programme from the $\sim 10^5$-neuron regime

to $\sim 10^6$ neurons, and the companion validity paper [5] extends the external-validation argument of Amos et al. [4] to the MeSH tree and citation-derived taxonomies.

3. Method

3.1. Problem statement and notation

Table 1: Notation used throughout.

Symbol	Meaning
N	number of input articles
V	number of MeSH features (vocabulary size)
nnz	nonzero features per input (MeSH per article) (mean = 11.1)
M	number of neurons (map units), $M = E^2$
E	edge of the square lattice
i, v	neuron index, feature index
W	codebook comprising $M \cdot V$ weights
x, w_i	input vector, prototype of neuron i
TA	inputs per BMU tile (16 in production)
σ	neighbourhood radius
QE, TE	quantisation error, topographic error

The BMU product contracts over V, not over the neuron count: the score block is $X_{N\times V}\, W^{\top}_{V\times M}$. Beware the BLAS collision - in GEMM's own $M{\times}K{\times}N$ naming, GEMM's M is this paper's N, GEMM's K is V in this paper, and GEMM's N is M here.

Let the corpus contain N inputs over V binary features, with each input x having a small number nnz of nonzero features (mean 11.1 in the present corpus). The map has M neurons arranged on a planar square lattice of edge E, so $M = E^2$. Each neuron i carries a prototype $w_i \in \mathbb{R}^V$. The BMU of x is

$$\mathrm{BMU}(x) \;=\; \arg\min_i \|x - w_i\|^2 \;=\; \arg\min_i \Big(\|w_i\|^2 - 2\langle x, w_i\rangle\Big), \qquad (1)$$

since $\|x\|^2$ is constant across neurons. For binary x the inner product $\langle x, w_i\rangle$ is a gather-sum over the input's nonzero features, so the BMU search reduces to, for each input, gathering the corresponding rows of the codebook and accumulating per-neuron partial sums and norms. To guarantee bit-reproducible results I sort each input's features in ascending order (fixing

the floating-point accumulation order), compute $\|w_i\|^2$ in the same order, and break ties by lowest neuron index. Under this exact-match design every kernel and layout returns identical BMUs.

3.2. The feature-major codebook

I store the codebook feature-major: W[v·M+i] places the M weights for feature v in a contiguous block. The consequence for the BMU search is that, when a warp processes the contribution of a single nonzero feature v, the M weights it must read are contiguous, so the loads coalesce into a small number of cache-line-aligned transactions. The node-major alternative W[i·V+v] places a neuron's V weights contiguously; a warp computing different neurons for the same feature then issues a strided gather across the full vocabulary stride, and because the codebook far exceeds the L2 cache, each gather is an uncoalesced HBM round-trip. Beyond coalescing, the feature-major layout enables reuse: the same feature column, once resident, serves every input in a processing tile that contains that feature (3.3). The codebook holds M·V weights; I store them in IEEE half precision (FP16, "__half") and convert to FP32 only for the dot-product accumulation, so the resident codebook is 2·M·V bytes. Because the BMU is an exact argmin, halving the storage precision halves the codebook footprint with no effect on the assignment. This FP16 codebook is what lets the largest maps fit: the 262,144-neuron codebook occupies ≈16 GB in FP16, within the 4090's 24 GB where both cuSPARSE layouts run out of memory (5.5), and the 1,048,576-neuron codebook ≈65 GB, within the H200's 141 GB (5.6). It is worth noting that the benefit is layout-dependent - the feature-major search gains 1.65× from FP16, the node-major search only 1.07× (5.1).

3.3. The SpMM-tile BMU kernel

The BMU kernel processes inputs in tiles of TA samples (the production kernel uses TA = 16; the microbenchmark of 5.1 used an earlier TA = 8). For each tile, the block first forms the union of the distinct features the tile's inputs touch - at most min(TA·nnz, V) features, roughly 180 at TA = 16 in the present setting. It then sweeps these feature columns once: each loaded weight is applied to all TA inputs that contain that feature, accumulating per-neuron partial dot-products and norms, and the argmin (BMU selection) is fused into the same sweep rather than run as a separate pass. The read complexity is therefore O((N/TA)·nnz·M + N·M), tracking the sparse lower bound N·nnz·M rather than the dense N·V·M of a naive sweep. The tile size

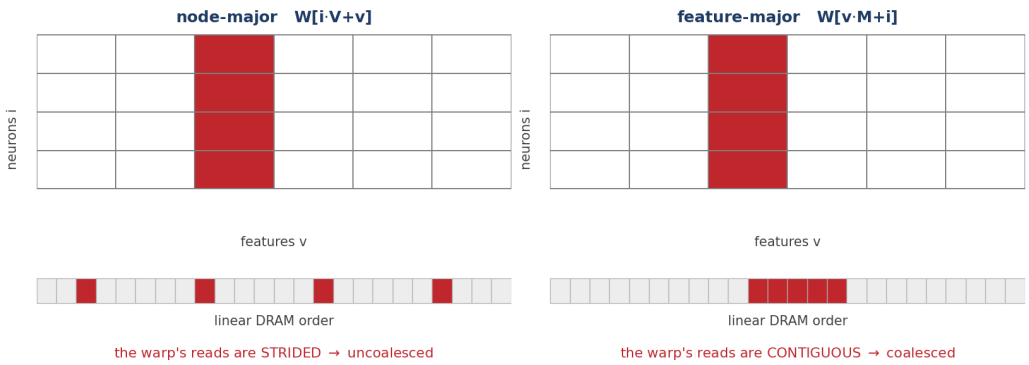

Figure 1: Codebook layout and the BMU access pattern. Highlighted are the M weights one warp reads for a single feature v. Node-major storage (W[i·V+v]) places a neuron's features contiguously, so reading one feature across neurons is strided in DRAM and the accesses cannot coalesce; feature-major storage (W[v·M+i]) places a feature's neurons contiguously, so the same reads are a single coalesced transaction - and the loaded column can be reused across a tile.

trades occupancy against per-tile work: smaller tiles create more blocks and better parallelism, larger tiles increase reuse but enlarge the per-tile feature union.

3.4. Factorised box-blur update, PCA initialisation, and the stopping rule

The neighbourhood update is best seen as a convolution. Once the batch has been scattered onto the lattice - accumulating, per cell, the sum of the assigned samples and their count - replacing each neuron's prototype with its neighbourhood-weighted mean amounts to blurring those two accumulator fields with the neighbourhood kernel and dividing one by the other. I perform the blur as a factorised (separable) box filter: a one-dimensional pass along the rows followed by one along the columns. Each pass is maintained as a running window sum, so every cell costs a constant two operations - add the element entering the window, subtract the one leaving it - regardless of the window's width [8]. A box pass is therefore O(M) and, crucially, independent of the neighbourhood radius σ: widening the neighbourhood only moves the running sum's endpoints, it never adds work per cell. Three successive box passes approximate a Gaussian neighbourhood by the central-limit theorem [21], so a fixed three passes in each direction reproduce a Gaussian-shaped kernel at fixed cost. The window half-width is σ rounded to whole cells and never smaller than one, and three passes of radius r give

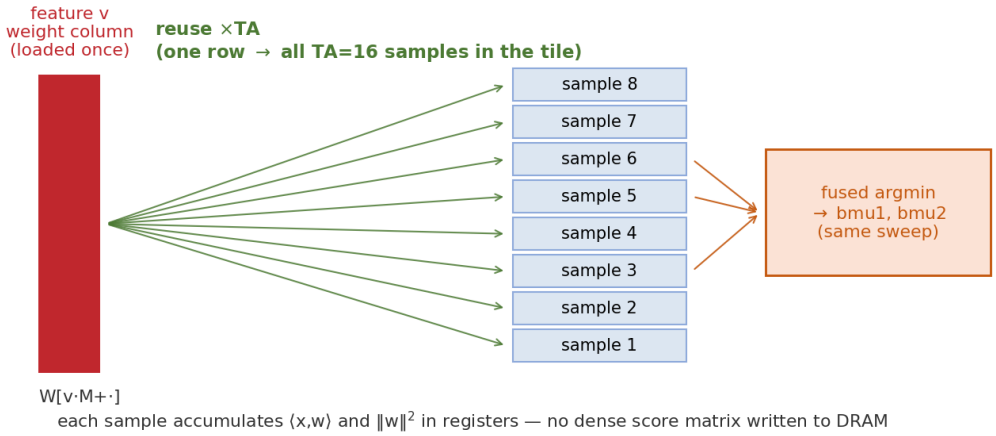

Figure 2: The SpMM-tile BMU kernel. Each feature column, loaded once, is reused across all TA samples of a tile; every sample accumulates its dot-product and norm in registers, so no dense N×M score matrix is written to DRAM (the round trip that dominates the cuSPARSE path, 5.7). The two best units per sample are found in the same fused sweep.

a kernel whose effective standard deviation is about $r + \frac{1}{2}$ cells: σ names the schedule variable, and the realised width is that much larger. This is what keeps the update flat in σ, and hence in map size (5.4). A direct Gaussian update costs about $O(M \cdot \sigma^2)$ because it re-sums the whole neighbourhood area at every neuron, and even a separable Gaussian costs $O(M \cdot \sigma)$, whereas the running-sum box blur is $O(M)$ for any σ. The saving is largest exactly where it matters, since σ starts near half the map edge and is largest on the biggest maps.

Four details matter for reimplementation. The box sums are unnormalised: the $(2r+1)$ constant cancels because the numerator and denominator fields receive the identical kernel, so an implementation that normalises one and not the other is silently wrong. The lattice boundary is clamped rather than toroidal, the window truncating at the edge identically in both fields, which leaves the ratio a proper weighted mean over the cells that remain. A neuron whose blurred denominator is zero is left unchanged rather than filled, which happens only when no sample landed anywhere within its neighbourhood support. And the accumulation and the blur are carried in FP32, the conversion to the FP16 codebook occurring only at the final write. The update itself is a pure replacement with no learning rate: every sample of the batch is weighted equally within the neighbourhood kernel each epoch.

The radius follows an exponential endpoint schedule from $\sigma_0 = 0.5{\cdot}\mathrm{E}$ down to a small $\sigma_{\min}$ (rate 0.3).

I initialise the codebook by projecting onto the leading principal components of the input (PCA initialisation). Under this schedule the progress variable is the epoch index itself, so the epoch count is set by the schedule and the plateau rather than by the initialisation: within SparseBin.SOM, PCA and random initialisation reach the plateau within two epochs of each other at every map size, with no systematic direction, and at the default initial radius $\sigma_0 = 0.5{\cdot}\mathrm{E}$ to statistically indistinguishable held-out quality. That equivalence is a property of an epoch-driven schedule and does not transfer to implementations whose radius is driven by measured progress (5.2).

PCA's value is therefore twofold and distinct from a raw epoch saving: it yields a deterministic, bit-reproducible result, and it licenses a smaller initial neighbourhood. Halving σ_0 to $0.25{\cdot}\mathrm{E}$ under PCA improves held-out quantisation error by roughly 1% (up to about 2% on the largest maps) while trimming a few epochs, whereas the same reduction degrades quality under random initialisation, which depends on the wide-neighbourhood ordering phase that a pre-ordered PCA map renders largely redundant.

Batch SOM training has no intrinsic length - too few epochs leave the map under-organised, too many merely waste compute - and the natural number depends on map size, initialisation, and the annealing schedule, so any fixed epoch budget is arbitrary. Training instead passes through two regimes: a broad ordering phase (large σ) that fixes the global topological arrangement of the lattice, and a fine-tuning phase (small σ) in which that arrangement is settled and only local quantisation still improves.

In the fine-tuning phase a plateau in map distortion is a reliable indicator of convergence [9], so I stop when it flattens rather than after a preset number of epochs. The monitored quantity is the Kaski-Lagus distortion: for each monitored sample, the distance from the input to its best-matching prototype plus the length of a lattice path from the best to the second-best unit, each edge of that path measured as the input-space distance between adjacent prototypes and the path capped at eight edges; the reported value is the mean over the monitored set.

Training stops once the map has entered the refinement regime ($\sigma \leq \max(1,\, 2\sigma_{\min}) = 1$, at every map size) and the relative change in that quantity stays below 0.1% (a relative change < 0.001) for three consecutive epochs, which under the configuration used throughout occurs after 20 to 29 epochs depending on map size. A separate check that topographic error is at most

0.5 decides whether a run is labelled converged; it does not affect when training halts. That is what makes the epoch count adapt to each map rather than being fixed in advance (contrast MedSOM's fixed 20 epochs, 5.5).

This brings me to the observation that underpins the evaluation. Because the BMU is an exact argmin, it is identical for any correct implementation and any codebook layout; the assignment phase cannot, even in principle, change clustering quality. Every difference in QE, TE, or dead-unit fraction between two SOM implementations must therefore come from the update axis - the neighbourhood kernel and σ schedule - not from the BMU. I exploit this in 5.3 to show that, once a baseline is given the box-blur update, its quality converges to that of SparseBin.SOM, confirming that the layout/BMU speed-up costs nothing in quality.

Algorithm 1 MedSOM batch SOM - node-major, as published [4]

Input:	X - N sparse-binary vectors over V features; M = E^2 neurons (planar square lattice)	
Data:	W - codebook of M dense node vectors, node-major W[i·V+v], FP32	a
1	randomise W	
2	for epoch e = 1 … 20 do	d
3	$R = \max(1, \lfloor (E/2) \,/\, 1.7^{e-1} \rfloor)$ // neighbourhood radius; $E/2$ at $e = 1$. Gaussian width is $R/2$	
4	L2-normalise every node vector of W	b
5	for each sample, each neuron i: dot $\langle x, w_i \rangle$ // node-major strided gather	a
6	rank units by $\|x\|_0 - 2 \cdot dot_i$ // $\|w_i\|^2$ omitted: step 4 makes it 1 for every unit	b
7	bmu1, bmu2 two smallest	
8	accumulate per-node numerators / denominators	
9	W weighted mean, Gaussian $l(e) = \exp(-\|r_i - r_c\|_1^2 / 2(R/2)^2)$ // no radius cutoff: every node sweeps every article, so cost is independent of R	c
10	TE fraction with bmu1, bmu2 non-adjacent	
11	end for	
12	return the codebook with the lowest TE over the 20 epochs	d

Algorithm 2 SparseBin.SOM: feature-major sparse-binary batch SOM

Input:	X - N sparse-binary vectors over V features; M = E^2 neurons (planar square lattice)	
Data:	W - codebook of M×V weights, feature-major W[v·M+i], FP16	a
1	initialise W by projection onto the leading principal components (PCA)	
2	for epoch e = 1, 2, … until the Kaski-Lagus plateau do	d
3	σ exponential endpoint schedule $\sigma_0 = 0.5 \cdot E \to \sigma_{\min}$	
4	for each tile of TA samples do // SpMM-tile: coalesced + reuse	a
5	load the distinct feature columns the tile touches	
6	for each neuron i: accumulate $\langle x, w_i \rangle$ and $\|w_i\|^2$ // exact, keeps $\|w_i\|^2$	b
7	bmu1, bmu2 two smallest ($\|w_i\|^2 - 2\langle x, w_i \rangle$) // fused argmin	
8	accumulate num, den (feature-major)	
9	blur num and den, three box passes per axis of half-width $\lfloor \sigma \rceil$; W num / den // σ-independent	c
10	QE mean($1 - \cos(x, w_{\text{bmu1}})$); TE fraction non-adjacent	
11	if $\sigma \leq 1$ and $\Delta KL < \varepsilon$ for three consecutive epochs then break // KL = Kaski-Lagus distortion	d
12	return the trained codebook W and the BMU of every sample	

It is instructive to set the two batch SOMs side by side. Algorithms 1 and 2 share the same skeleton - anneal a neighbourhood, assign every sample to its best-matching unit, move each neuron toward the mean of its neighbourhood, repeat - but differ in four places that, between them, account for essentially all of the speed, scaling, and reproducibility differences reported in 5. The steps that differ are shaded and keyed a-d to the notes beneath the algorithms, with the same tint used for the same note in both algorithms so that corresponding steps line up; the unshaded steps are common to both.

Algorithm 1. MedSOM, the node-major MEDLINE batch SOM of the earlier published work [4]. Shaded steps, keyed a-d, are those that differ from Algorithm 2; tints match across the two algorithms. Step 5 is the strided, uncoalesced gather drawn in the left-hand panel of Figure 1.

Algorithm 2. The feature-major method introduced here. Shaded steps, keyed a-d, mark the four high-impact differences from Algorithm 1; tints match Algorithm 1. Step 4 is the coalesced feature-column read drawn in the right-hand panel of Figure 1, and steps 4-7 are the tile sweep of Figure 2, in which one loaded column serves every sample of the tile and the two best units are taken without materialising a score matrix. See the notes beneath.

a Memory layout and BMU access - the decisive performance change. MedSOM stores the codebook node-major and gathers it with a strided, uncoalesced pattern that thrashes the L2 cache; the feature-major layout makes each feature column a coalesced read and reuses it across the TA samples of a tile. This single change is worth 4.5-8.5× on the BMU search when nothing else is varied (5.1), and is the reason the BMU dominates training time at every scale (5.4).

b Distance calculation. MedSOM L2-normalises the whole codebook every epoch so it can rank units by $\|\mathrm{x}\|_0 - 2\langle \mathrm{x},\mathrm{w}\rangle$, dropping $\|\mathrm{w}\|^2$; this normalisation is destructive (the learned weights are rescaled each epoch) and renders its QE non-comparable across implementations. Algorithm 2 carries an exact $\|\mathrm{w}_i\|^2$ in the fused BMU sweep, so the codebook is preserved and QE is directly interpretable.

c Neighbourhood update. MedSOM applies its Gaussian with no radius cutoff, so every node sweeps every article on every epoch and its per-epoch cost is independent of the radius and maximal at all radii. The cuSPARSE baseline's Gaussian does scale with the radius, growing 133-fold across a sixty-fourfold increase in neurons, whereas the factorised three-pass box blur is σ-independent and grows eightfold. This is why the update advantage grows with map size - from parity at 32×32 to 14.5× at 256×256 (5.4).

d Stopping rule. MedSOM runs a fixed 20 epochs and keeps the lowest-TE codebook post hoc; Algorithm 2 stops adaptively on a Kaski-Lagus plateau rather than after a fixed epoch budget (3.4).

Fidelity of Algorithm 1 to MedSOM. Algorithm 1 describes both the published MedSOM [4] and the implementation benchmarked here: the two share the same training-path source, the port having touched only input handling and build plumbing. Three details are compressed for readability. First, the radius schedule carries a floor at 1 and is truncated to an integer, so at $E = 350$ the sequence over 20 epochs is 175, 102, 60, 35, 20, 12, 7, 4, 2 and then 1 for the remaining eleven epochs. Second, the published schedule is printed as $175/(1.7)^{\text{epoch}}$ [4], but the epoch counter is zero-indexed in the source, so the first epoch uses the full $E/2$ rather than $E/2$ divided by 1.7; Algorithm 1 follows the code.

Third, step 12's selection of the lowest-topographic-error codebook is performed post hoc over saved checkpoints rather than by the program itself, and the error that gates a checkpoint is measured before that epoch's update while the file is written after it. Separately, MedSOM runs in FP32 throughout, where Algorithm 2 stores its codebook in FP16 and accumulates in FP32; because the benefit of FP16 is layout-dependent, the consequences for the timing comparison are bounded rather than removed, and are quantified in 5.5.

4. Experimental setup

4.1. Hardware

All comparisons between implementations were measured on a single NVIDIA RTX 4090 (AD102; 24 GB VRAM, 72 MB L2, ∼1 TB/s peak bandwidth, 82.6 TFLOP/s FP32), driver 610.43.02, CUDA 12.8. The largest map in the size sweep (1024×1024, 5.6) was trained on an H200 SXM (141 GB VRAM) rented on demand, under the earlier adaptive σ schedule; it is labelled as such wherever it appears. The somoclu CPU baseline (version 1.7.6, build 1.7.6-11-g63895f4, sparse CPU kernel) runs on the host over 28 threads. Provenance - submodule commits, resolved configuration hash, GPU, driver, and CUDA version - is recorded per artifact by the reproduction pipeline.

Use of AI tools in the research process. Generative AI assistance (Anthropic Claude) was used throughout the research process: implementing and running the training and profiling experiments, building the reproduction pipeline, writing the analysis and figure-generation scripts, and performing

the statistical analyses reported in 5. All figures are derived directly from the released data by reproducible scripts, which are part of the code release (8); no figure contains AI-generated content that is not computed from the underlying measurements. Every experimental design decision, every analysis choice, and every claim made in this paper was directed and approved by the author, who takes responsibility for them.

4.2. Corpus and data format

Articles were drawn from the PubMed 2026 baseline, a fixed snapshot, and retained only if they carried at least five MeSH descriptors, which yields 29,903,261 articles over 30,766 descriptors. The resulting corpus is archived at `https://doi.org/10.5281/zenodo.20770707` (8). Records outside the MeSH-indexed MEDLINE subset - PubMed Central deposits, in-process and ahead-of-print citations, books - carry no descriptors and are therefore never candidates; the floor of five is what additionally excludes thinly indexed MEDLINE records. The mean of 11.1 descriptors per article follows from that floor. The corpus is stored in .sbcsr (sparse-binary compressed-sparse-row), a bespoke on-disk container introduced for this work rather than a community standard; the layout inside it is ordinary CSR, with the value array omitted because every stored element is 1. Each article is a row holding the sorted integer indices of the MeSH descriptors present in it, in the standard CSR layout - a row-pointer array giving where each article's index list begins, plus one concatenated array of those descriptor indices. The entry values are left implicit, because in a binary incidence matrix every stored element is a 1. Recording presence alone (about eleven indices per article) rather than a dense 30,766-element vector, the entire 29.9-million-article corpus occupies roughly 750 MB on disk and streams directly into the GPU kernels without decompression.

4.3. Baselines and comparators

Four implementations are compared, and are named in the tables as follows. **SparseBin.SOM** is the feature-major sparse-binary method introduced here (Algorithm 2), and **SparseFloat.SOM** is the same design over real-valued sparse inputs. **cuSPARSE.SOM** is the baseline described below, reported in whichever codebook ordering is relevant and labelled feature-major or node-major accordingly. **MedSOM** is the prior CUDA implementation of Algorithm 1, and **somoclu** is the multicore CPU library. The repositories that hold these implementations carry their development names

Table 2: Corpus statistics.

Property	Value
Source	PubMed 2026 baseline (1,334 files, complete)
Filter	Articles with ≥ 5 MeSH descriptors
Articles (N)	29,903,261
Vocabulary (V)	30,766 MeSH descriptors
Nonzeros	332,436,043 (mean 11.12 per article, sd 4.60; density $\approx 0.036\%$)
Format	.sbcsr sparse-binary CSR; anonymised (no PMIDs); released under CC0 at doi:10.5281/zenodo.20770707

rather than the labels used here, and the release README gives the correspondence.

- cuSPARSE.SOM - a same-GPU cuSPARSE baseline (cusparseSpMM; [16]), run in both feature-major (ORDER_ROW) and node-major (ORDER_COL) orderings.
- MedSOM - our prior CUDA SOM, ported to Linux with native .sbcsr loading; serial per-node update and an approximate Euclidean metric (it L2-normalises the codebook each epoch, then ranks by nnz $-$ 2·dot).
- somoclu - the multicore sparse-CPU kernel, the only somoclu kernel viable at V $\approx$ 30k.

Fairness is controlled by using the identical corpus and map sizes, matching the per-size epoch budget and σ endpoints where the comparison is on training cost, and scoring all codebooks with a single shared cosine evaluator. Documented asymmetries - kernel shape (box-blur vs Gaussian), GPU vs CPU, initialisation (PCA here, uniform random in the cuSPARSE baselines, which have no PCA option), and the driver of the σ schedule (the epoch index here, measured progress in the baselines) - are disclosed rather than hidden, and are noted where they bear on a specific comparison.

4.4. Quality metrics

Cross-implementation quality is scored by: QE = mean $1 - \cos(x, \mathrm{BMU}_1)$; TE = fraction of inputs whose first and second BMUs are non-

adjacent (Chebyshev distance > 1); and the dead-unit fraction. Every implementation is scored the same way, by a single external evaluator reading its trained codebook, rather than by trusting the error each implementation prints for itself. This matters because those self-reported figures are computed under whatever internal convention the implementation happens to use: MedSOM, for instance, L2-normalises its codebook every epoch, so its internal quantisation error is measured against rescaled weights on a metric that has drifted toward cosine, and is numerically incomparable with an error computed from unnormalised prototypes. Self-reported values are therefore used only to track progress within a single implementation, never to compare one implementation against another.

5. Results

5.1. Codebook layout: feature- versus node-major

The cleanest test of the layout argument holds everything else fixed. Because the cuSPARSE baseline exists in both layouts - the same implementation, the same precision, the same update and stopping rule, differing only in whether the codebook is stored node-major or feature-major - the ratio between them isolates the effect of memory layout alone (Table 3). Feature-major storage accelerates the best-matching-unit search by 8.5× at 32×32, and the advantage remains between 4.5× and 8.1× across the rest of the ladder. The mechanism is the one set out in 3.2: reading one feature's weights across neurons is a coalesced transaction under feature-major storage and a strided gather under node-major, and at a codebook far larger than the L2 cache every uncoalesced gather becomes an HBM round trip.

Table 3: Effect of codebook layout in isolation.

Edge	Node-major BMU/epoch (s)	Feature-major BMU/epoch (s)	Layout speed-up
32	4.04	0.478	8.5×
64	16.31	2.02	8.1×
128	65.65	8.72	7.5×
256	289.66	63.84	4.5×

cuSPARSE.SOM in its node-major and feature-major variants, full corpus, matched update (box+KL), FP16, mean of five seeds. The two columns differ only in codebook layout, so the ratio is attributable to layout alone.

A second, independent observation confirms that layout - not precision - is what limits the node-major path. Halving the codebook to FP16 accelerates

the feature-major kernel by 1.65×, close to the 2× its halved read volume would predict, but accelerates the node-major kernel by only 1.07×. The node-major search is not bandwidth-bound at all: it is limited by the latency of dependent, uncoalesced gathers, so giving it fewer bytes to move barely helps. For that layout the access pattern is the whole problem, which is why I treat the layout rather than the storage format as the primary design decision.

5.2. Equal-quality comparison with cuSPARSE: a crossover, and a capacity wall

I next compare SparseBin.SOM against the stronger of the two cuSPARSE.SOM baselines at matched quality, letting each run to its own Kaski-Lagus plateau rather than a fixed budget (Table 5). Held-out quantisation error agrees to within 0.5% at every map size, and to within 0.07% at the two largest - so whatever else differs, the two implementations are converging to maps of the same representational quality. On topographic error and dead-unit fraction SparseBin.SOM is slightly ahead (at 256×256, TE 0.033 versus 0.037 and 10.7% versus 17.4% dead), so the comparison is equal-or-better rather than merely equal.

Against that fixed quality, the timing is not a single multiple but a crossover (Table 4). Wall-clock time throughout this paper means elapsed real time, as distinct from summed kernel time or CPU time; where the scope is a whole training run, as here, it is the total from initialisation to the Kaski-Lagus plateau, summed over however many epochs that run needed rather than cost per epoch. The two differ because the runs converge in different numbers of epochs. At 32×32 the cuSPARSE path is 2.8× faster; at 64×64 the two are at parity; from 128×128 onward SparseBin.SOM pulls ahead, by 1.5× at 128×128 and 2.6× at 256×256. Two effects compound in SparseBin.SOM's favour as the map grows: the per-epoch BMU search becomes faster (from 0.23× of the baseline's speed at 32×32 to 1.53× at 256×256), and the present runs converge in fewer epochs (25 against 40 at 256×256).

The epoch difference is not a property of the layout, and is worth attributing plainly. The two arms differ in three disclosed respects that bear on epoch count: SparseBin.SOM is PCA-initialised while the baseline draws its codebook uniformly at random; SparseBin.SOM anneals on an epoch-driven deterministic schedule while the baseline's radius is driven by measured progress, so it holds the radius wide for as long as the map keeps improving; and, more marginally, they evaluate the stopping metric over

monitored sets of different size (the first 100,000 samples here, 200,000 in the baseline). Both arms test the same fully weighted Kaski-Lagus distortion, and the one remaining gate difference - the path-term guard - decides only whether a run is labelled converged, not when it halts.

A random start leaves more ordering work to do, and under a progress-driven radius that work converts directly into epochs, which is why the gap widens with map size (one epoch at 32×32, roughly fifteen at 256×256). The per-epoch BMU advantage is attributable to layout; the epoch advantage is attributable to initialisation and schedule, and the wall-clock figures in Table 4 combine the two. At 512×512 the comparison ends - both cuSPARSE layouts exhaust the 24 GB card, while SparseBin.SOM trains on. The honest summary is therefore that the bespoke kernel is not universally faster; it is faster where large maps are wanted, and it is the only one of the two that reaches them.

Table 4: The equal-quality crossover.

Edge	Wall to plateau (s)		Epochs to plateau		Speed-up
	cuSPARSE.SOM	SparseBin.SOM	cuSPARSE.SOM	SparseBin.SOM	(SparseBin.SOM)
32	18.8	52.8	23	22	0.36×
64	75.5	71.8	27	20	1.05×
128	366.7	249.3	34	25	1.47×
256	2,758.2	1,061.5	40	25	2.60×
512	OOM	5,427.1	-	29	-

Wall-clock time from initialisation to each run's own Kaski-Lagus plateau, so the totals absorb both per-epoch cost and the number of epochs required. A speed-up below 1 means cuSPARSE is faster. The crossover falls between 32×32 and 64×64, and the margin then widens with map size, driven by both a faster per-epoch BMU search and earlier convergence. At 512×512 both cuSPARSE layouts exhaust the 24 GB card, so no comparison is possible. Held-out QE agrees to within 0.5% at every size (Table 5).

5.3. Quality is set by the update, not the BMU

That the two implementations converge to the same quality is not a coincidence, and the reason is worth isolating. Giving the cuSPARSE baseline each of the update rules in turn (Table 6) shows that quality tracks the neighbourhood kernel and the stopping rule, not the search. Under a Gaussian neighbourhood the baseline has a topographic error between 0.15 and 0.23; replacing it with three box passes alone drops TE to 0.013-0.033, and adding the Kaski-Lagus stop brings quantisation error to within 0.5% of

Table 5: Equal-quality comparison, full corpus.

Edge	Model	Wall (s)	BMU/ep (s)	Epochs	QE (held)	TE	Dead %
32×32	SparseBin.SOM	52.8	2.296	22	0.5241	0.010	9.4
32×32	cuSPARSE.SOM	18.8	0.478	23	0.5264	0.015	9.8
64×64	SparseBin.SOM	71.8	3.432	20	0.4899	0.018	8.7
64×64	cuSPARSE.SOM	75.5	2.024	27	0.4885	0.021	10.8
128×128	SparseBin.SOM	249.3	9.653	25	0.4518	0.027	9.3
128×128	cuSPARSE.SOM	366.7	8.722	34	0.4515	0.029	12.9
256×256	SparseBin.SOM	1,061.5	41.79	25	0.4152	0.033	10.7
256×256	cuSPARSE.SOM	2,758.2	63.84	40	0.4154	0.037	17.4
512×512	SparseBin.SOM	-	184.8	-	-	-	-
512×512	cuSPARSE.SOM	OOM	OOM	-	-	-	-

All runs use the feature-major cuSPARSE baseline, matched update (box+KL) and FP16 throughout, each run stopped on its own Kaski-Lagus plateau; mean of five seeds. Held-out QE agrees within 0.5% at every size (0.07% at 128×128 and 0.05% at 256×256). The wall-clock time advantage crosses over between 64×64 and 128×128; both cuSPARSE layouts exhaust 24 GB at 512×512, where the 512×512 BMU figure for SparseBin.SOM comes from the fixed-epoch efficiency sweep (5.5). BMU, best-matching unit; QE, quantisation error; TE, topographic error; OOM, out of memory (the run could not be allocated on the 24 GB card).

SparseBin.SOM at every size. Because the best-matching unit is an exact argmin, identical in every one of these runs, the quality differences can only come from the update - exactly as the exact-argmin argument predicts. The layout speed-up of 5.1 is therefore free of any quality cost.

5.4. The update phase, and where training time actually goes

The factorised box blur was introduced (3.4) to make the update cost independent of the neighbourhood radius, and the measurements bear this out (Table 7). The box-blur update rises from 0.031 to 0.248 seconds per epoch as the map grows from 32×32 to 256×256 - an eightfold increase for a sixtyfourfold increase in neurons - while the cuSPARSE path's update, which re-sums a neighbourhood whose area grows with σ^2, rises from 0.027 to 3.60 seconds. The bespoke update starts on par at the smallest map and is 14.5× cheaper by 256×256, so the feature-major design does not merely avoid harming the update phase, it improves it, and the margin widens with map size exactly as the O(M) versus O(M·σ^2) argument predicts.

Set against the BMU search, the update is almost negligible (Figure 3). The best-matching-unit search consumes 98.7% of training wall-clock time at 32×32, rising monotonically to 99.5% at 512×512, while the update never

Table 6: Effect of the update rule on quality.

Edge	Impl	Update	TE	QE (held)	Dead %
32	cuSPARSE.SOM (feature-major)	Gaussian	0.152	0.4889	3.5
32	cuSPARSE.SOM (feature-major)	box	0.013	0.5289	9.5
32	cuSPARSE.SOM (feature-major)	box+KL	0.015	0.5264	9.8
32	SparseBin.SOM	box+KL	0.010	0.5241	9.4
128	cuSPARSE.SOM (feature-major)	Gaussian	0.214	0.4222	3.9
128	cuSPARSE.SOM (feature-major)	box	0.028	0.4601	12.8
128	cuSPARSE.SOM (feature-major)	box+KL	0.029	0.4515	12.9
128	SparseBin.SOM	box+KL	0.027	0.4518	9.3
256	cuSPARSE.SOM (feature-major)	Gaussian	0.229	0.3896	6.2
256	cuSPARSE.SOM (feature-major)	box+KL	0.037	0.4154	17.4
256	SparseBin.SOM	box+KL	0.033	0.4152	10.7

Quality is governed by the update, not the BMU. The Gaussian rows show the baseline's native update; the box rows substitute the three-pass box blur; box+KL adds the Kaski-Lagus stop. TE falls by an order of magnitude with the box blur alone, and box+KL brings the baseline to within 0.5% of the held-out QE here at every size. Mean of five seeds.

Table 7: Neighbourhood-update cost per epoch.

Edge	SparseBin.SOM update (s/ep)	cuSPARSE.SOM update (s/ep)	Ratio	SparseBin.SOM BMU (s/ep)
32	0.031	0.027	0.9×	2.296
64	0.037	0.102	2.8×	3.432
128	0.082	0.558	6.8×	9.653
256	0.248	3.599	14.5×	41.793

Matched update rule (box+KL), full corpus, mean of five seeds. The radius-independent box blur grows 8× across a 64× increase in neurons; the cuSPARSE path's grows 133×. The rightmost column gives the BMU cost for scale - the update is a small fraction of it throughout.

exceeds 1.3%. Optimising the codebook layout is therefore not one improvement among several; it is the only part of the computation whose cost matters at scale.

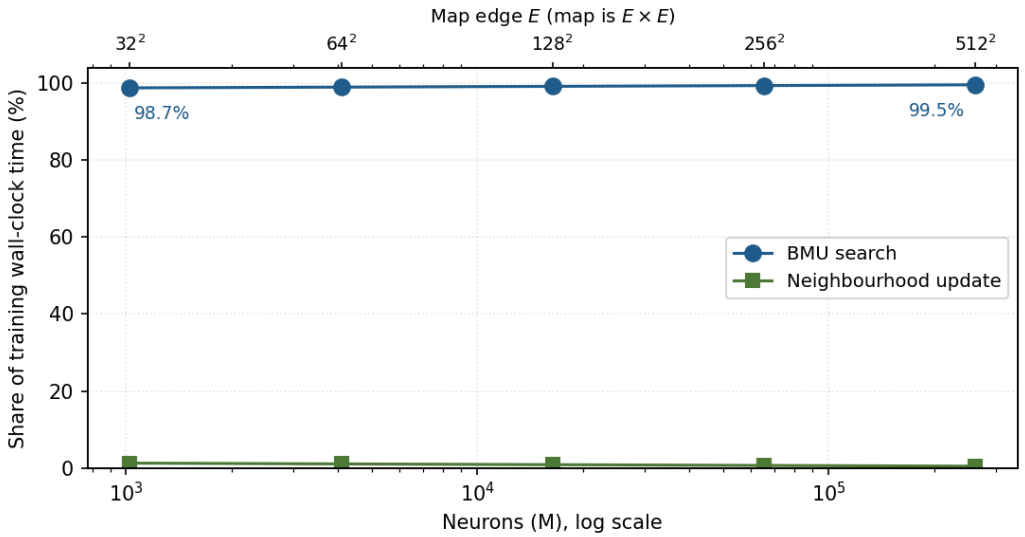

Figure 3: Share of training wall-clock time spent in the BMU search versus the neighbourhood update, across the size sweep (log neuron axis). The BMU rises from 98.7% of wall-clock time at 32×32 to 99.5% at 512×512, while the factorised box-blur update never exceeds 1.3%.

5.5. The efficiency chain, and the capacity wall

To place the implementation against the alternatives on a strictly like-for-like footing, I fixed the work - 20 epochs over the full corpus, the same split, the same map - and measured wall-clock cost for every implementation that could run at all (Table 8, Figure 4). At 128×128 the ordering spans nearly four orders of magnitude. The sparse-binary kernel needs 9.9 seconds per epoch. The sparse-float variant of the same design needs 27.0, so the binary specialisation is worth 2.1-2.8× across the ladder, though part of that is the halved FP16 codebook rather than binariness as such. The cuSPARSE feature-major baseline is competitive here at 9.3 seconds; its node-major counterpart needs 66.5. MedSOM, the CUDA implementation behind our earlier MEDLINE atlases, needs 792.8 seconds per epoch - 82× SparseBin.SOM - and somoclu, the best available multicore CPU library, needs 6,168.5 seconds on 28 threads, 621× SparseBin.SOM.

Three qualifications matter for reading this chain honestly. First, the advantage over the CPU library is not a fixed multiple but a trend that climbs steeply and then flattens: 85× at 32×32, 349× at 64×64, 621× at 128×128 and 655× at 256×256 - the last step adding only 5.5%. Two effects compound. somoclu's per-epoch cost grows super-linearly in neurons - 6.2×, 4.9× and 4.5× for successive quadruplings - while SparseBin.SOM grows sub-linearly at the smallest maps, where a 4090 is barely occupied, accelerating through 1.5×, 2.8× and 4.3×. The two rates converge near 4.3-4.5× by 256×256, which is why the ratio plateaus at roughly 650× rather than continuing to climb. The gap widens with map size and settles near three orders of magnitude rather than starting there. I describe the GPU-versus-CPU advantage as approaching three orders of magnitude at the map sizes that matter, and give the whole curve rather than a single figure, since any one number is an artefact of the size at which it was taken.

Second, the MedSOM figure is a timing comparison only: its per-epoch cost is measured on the same corpus and map, but I make no quality claim for it, since its destructive per-epoch normalisation renders its quantisation error non-comparable (2.2).

Third, and most important for attribution, these are end-to-end system comparisons rather than layout results: MedSOM and somoclu differ from SparseBin.SOM simultaneously in codebook layout, in neighbourhood update - per-node loops against the factorised box blur - and in precision, both being FP32 by construction.

The 82× and 621× figures therefore bundle all three, and should not be read as evidence for the layout argument; the isolated layout effect is the 4.5-8.5× of 5.1, measured within a single implementation with precision and update rule held fixed. Not every difference favours SparseBin.SOM: MedSOM's per-epoch normalisation makes its own search cheaper, so that difference works against the ratio rather than inflating it.

The precision component can be bounded, though not cleanly removed, because the benefit of FP16 is itself layout-dependent (5.1). Matching downward, by running SparseBin.SOM at FP32, would forfeit its 1.65× and leave roughly 50×; matching upward, by giving a node-major implementation FP16, would buy it little - 1.07× for the node-major cuSPARSE kernel - and leave roughly 77×. A single precision-matched figure therefore does not exist; the ratio lies near 50-77×, and the width of that interval is itself a consequence of the layout effect this paper argues for. The final row of the table is the one that most constrains practice - at 512×512 both cuSPARSE

layouts and MedSOM exhaust the 24 GB card, and SparseBin.SOM is the only one that runs.

Table 8: Per-epoch training cost at matched work.

Implementation	32×32	64×64	128×128	256×256	512×512
SparseBin.SOM	2.4	3.6	9.9	42.2	186.3
cuSPARSE.SOM (feature-major)	0.7	2.3	9.3	67.0	OOM
SparseFloat.SOM	4.9	8.6	27.0	112.9	480.4
cuSPARSE.SOM (node-major)	4.2	16.6	66.5	294.6	OOM
MedSOM (prior CUDA SOM)	37.6	197.8	792.8	3,209.9	OOM
somoclu (CPU, 28 threads)	202.9	1,251.5	6,168.5	27,685	-

Seconds per epoch at matched work (20 fixed epochs, full 26.9 million-article training split, RTX 4090; somoclu on 28 CPU threads at steady state). All GPU implementations are FP16 except MedSOM and somoclu, which are FP32 by construction. At 512×512 only SparseBin.SOM runs; the others exhaust 24 GB. somoclu was measured on an uncontended run at all four sizes; when contention was tested at 64×64 it cost under 2%.

5.6. Map-size scaling: a power law with no elbow

The final experiment asks how quality scales with map size when nothing else varies. Every rung trains on the same full-corpus split with the same initialisation, precision, and stopping rule, doubling the lattice edge each time from 32×32 to 512×512 on the RTX 4090, and I add a 1,048,576-neuron map trained on an H200 (Table 9, Figure 5). Held-out quantisation error falls monotonically across the whole range, from 0.5241 to 0.3437, and the decline is a clean power law. Because the largest rung was trained under the earlier adaptive σ controller rather than the deterministic schedule used for the rest, I report the fit both ways. Across the five deterministic rungs the exponent is -0.0597 (95% CI -0.067 to -0.052) with $R^2 = 0.996$, a 4.05% improvement in QE per doubling of the neuron count; including the H200 rung, which extends the range to three full decades, gives -0.0615 (95% CI -0.067 to -0.056) with $R^2 = 0.996$, or 4.17% per doubling. The two agree within each other's confidence intervals, so the extra point extends the curve without bending it.

The absence of an elbow is the substantive result, and because it is a negative claim I tested it rather than reading it off the plot. On the five deterministic rungs, fitting a segmented model with a free breakpoint [15] improves the residual sum of squares from 5.80×10^{-5} to 9.81×10^{-6}. An F-test does not support that as a real improvement ($F = 4.91$, $p = 0.27$): the

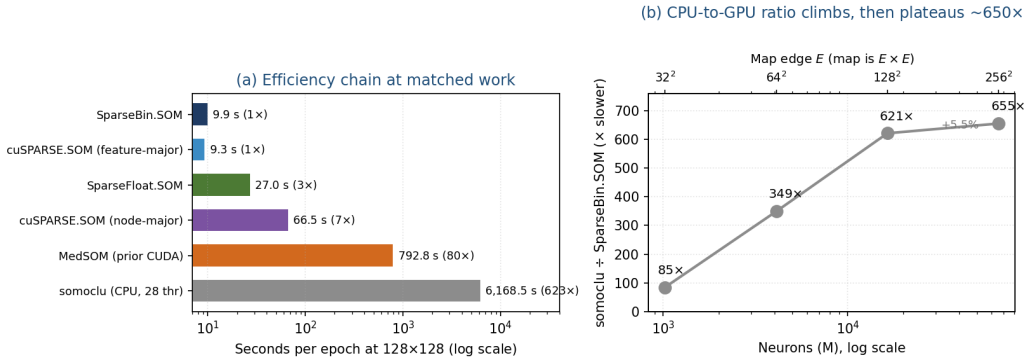

Figure 4: (a) Per-epoch training cost at 128×128 on the full corpus, matched at 20 epochs (log scale); the chain spans more than two and a half orders of magnitude from the best CPU library to the sparse-binary GPU kernel. (b) The CPU-to-GPU ratio is not a constant: it climbs steeply from 85× at 32×32 to 621× at 128×128, then flattens to 655× at 256×256, because somoclu's super-linear scaling and its sub-linear scaling converge as the map grows.

single power law is not rejected in favour of a two-regime fit at any interior knot. Adding the H200 rung the segmented model remains unsupported at the 5% level, but the margin narrows ($F = 15.5$, $p = 0.059$, best knot at 128×128), and I report this rather than the more convenient figure alone. Two observations settle how it should be read.

First, the extra point does not create the curvature: the five deterministic rungs alone select the same breakpoint and almost the same pair of slopes (-0.0528 and -0.0666, against -0.0527 and -0.0667 with the sixth point), so the largest map adds statistical power to detect a departure that is already present rather than introducing one.

Second, and decisively, the departure runs the wrong way to be an elbow. An elbow marks the point of diminishing returns - a shallower slope beyond the breakpoint, marking the size past which refinement stops paying. Here the slope steepens, from 3.59% per doubling below the knot to 4.52% above it: quantisation error improves faster as the map grows, not more slowly. Whatever mild two-regime structure the data contain therefore reinforces the conclusion rather than qualifying it, and no size in the range examined marks a point of diminishing return. Nor does the per-doubling gain show any sign of decaying toward a plateau: it runs 3.3%, 4.0%, 4.1%, 4.8%, 4.4% across the five doublings, drifting up rather than down, and never approaches the 0.5% threshold at which further refinement would cease to pay (Figure

5b). Two secondary measures confirm the maps stay healthy: topographic error rises gently from 0.010 to 0.045, and the dead-unit fraction from 9.4% to 20.0%, so four neurons in five remain active at the largest size.

Table 9: Held-out quantisation error across the size sweep.

Edge	Neurons	Epochs	Wall (s)	QE (held)	QE (Eucl)	TE	Dead %
32	1,024	22	52.5	0.5241	2.8181	0.010	9.4
64	4,096	20	71.5	0.4899	2.7592	0.018	8.7
128	16,384	25	249.1	0.4518	2.6876	0.027	9.3
256	65,536	25	1,062.5	0.4152	2.6105	0.033	10.7
512	262,144	29	5,427.1	0.3764	2.5204	0.042	14.7
1024*	1,048,576	44	35,495.3	0.3437	2.4312	0.045	20.0

A single full-corpus split (26,912,934 training articles), PCA initialisation, Kaski-Lagus stop; all six runs converged. Edges 32-512 were trained on the RTX 4090 under the deterministic exponential schedule. *The 1024×1024 rung was trained on an H200 SXM under the earlier adaptive σ controller and is reported separately for that reason; it uses the identical corpus split, initialisation, precision, and stopping criterion, and differs only in the σ decay path (see text). QE columns give held-out cosine QE and implementation-native Euclidean QE. Held-out QE falls monotonically across the whole range with no elbow.

At 1,048,576 neurons the largest map exceeds the biggest self-organising map in the peer-reviewed record - the roughly 1.0 million-node WEBSOM patent map [12] - and does so over a corpus four times larger, 29.9 million articles against 6.8 million patent abstracts, and ∼30,000 item input vectors against ∼500. I am not aware of a larger trained SOM. The run was trained under the earlier adaptive σ controller, so it is not schedule-matched to the rest of the ladder; and it is a single run, as are all rungs, so the sweep carries no interval. That is a weaker limitation than it sounds: PCA-initialised full-batch training on a fixed split is deterministic, so each rung is exact rather than a draw from a distribution, and re-running it reproduces the same value.

5.7. Where the advantage comes from, and where it does not

Since the BMU search dominates training cost, I profiled the whole search phase with Nsight Compute [17] at two map sizes, summing every kernel each implementation runs - for the cuSPARSE path the codebook norm, the sparse-dense product and the argmin reduction; for SparseBin.SOM the single fused kernel that does all three (Table 10, Figure 6). The result overturns the mechanism I had previously proposed, and I report it as such. The natural explanation for a sparse-binary kernel's speed is that it simply moves

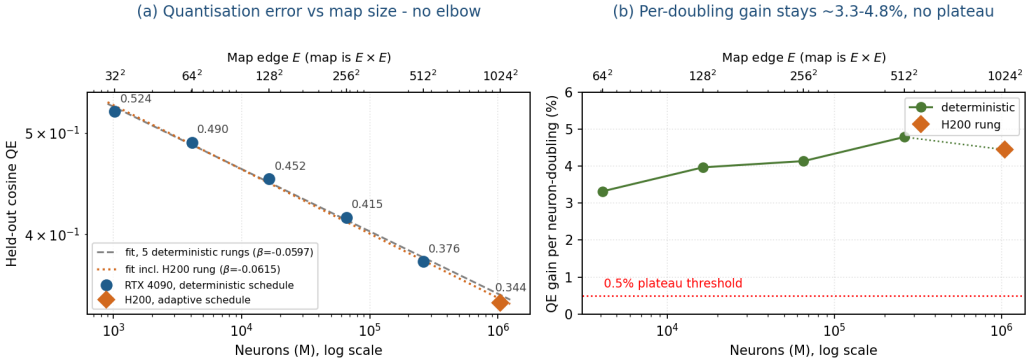

Figure 5: (a) Held-out cosine QE against neuron count (log-log) from 32×32 to 512×512 on one full-corpus split, with the fitted power law. (b) QE improvement per neuron-doubling, drifting up from 3.3% to 4.8% and never approaching the 0.5% plateau threshold.

fewer bytes: recording presence rather than values, and holding the codebook in half precision, ought to cut the DRAM traffic on which a bandwidth-bound search depends. An earlier version of this work reported exactly that, a 3.85× reduction in DRAM moved. The figure does not survive precision-matching. It compared an FP16 kernel against an FP32 baseline, so most of the gap was the precision difference rather than the representation, and with both implementations storing the codebook in half precision the traffic advantage disappears.

What replaces it is a crossover rather than a direction. At 128×128 the fused kernel moves slightly less data than cuSPARSE feature-major - 2.82 TB against 3.10 TB per epoch, some 9% less - while at 256×256 it moves substantially more, 18.97 TB against 13.85 TB, or 37% more. Sustained bandwidth crosses in the same place and the opposite way: at 128×128 the cuSPARSE path streams faster (35.3% of peak against 29.0% here), whereas at 256×256 SparseBin.SOM streams more than twice as fast (45.2% against 21.5%, with node-major managing 7.8%). Neither traffic volume nor bandwidth efficiency is a fixed property of either design.

The per-kernel breakdown identifies the mechanism directly, and it is not the one I expected. Because the cuSPARSE path computes scores with a sparse-dense product, it must materialise the dense score block in memory and read it back to find each sample's best unit. That costs 0.86 TB written and 0.89 TB read at 128×128, and 3.36 TB and 3.56 TB at 256×256 - between a half and rather more than a half of everything that path moves.

The fused kernel accumulates distances in registers and takes its two best units in the same sweep, writing no score block at all. The decisive observation is what that read-back costs in time. At 256×256 the sparse-dense product streams at 67.7% of Nsight Compute's measured sustained peak (≈982 GB/s) and finishes 10.27 TB in 15.4 seconds; the reduction that follows it streams at 7.7% of the same peak and takes 47.3 seconds to move 3.57 TB. The reduction is thus a quarter of the traffic and three-quarters of the time (Figure 6c). Reading a dense score block back is a poorly-localised access over a structure far larger than cache, and it is precisely this pass that fusion removes.

The advantage at large maps therefore comes from not materialising the score matrix rather than from moving less data, which vindicates the fusion rationale of 3.3 rather than the traffic argument I had built on top of it. A separate instruction-level profiling addendum, using the same six captures with an extended counter set, locates the fused kernel more precisely still: it executes essentially no floating-point multiply, reaches only 30 to 46% of the nearest ceiling at every level of the memory hierarchy and no more than 32% of warp-issue peak, and is therefore latency-limited rather than bandwidth- or compute-bound, its 121 registers per thread holding occupancy to a third. That profiling requires a Nsight Compute-enabled image and elevated capabilities that the reproduction container deliberately does not carry, so it is released as a separate repository, `https://github.com/mongrolwarrior/sparsesom-roofline-addendum` at tag `v1.0`, rather than as part of the pipeline of 8; a written addendum accompanies this article as an ancillary file.

6. Discussion

Taken together, the results cohere around a single mechanism, though not the one I first proposed. Training a self-organising map at this scale is almost entirely a problem of moving the codebook through memory: the best-matching-unit search reads it on every epoch, and at a codebook far larger than the cache it is the access pattern, not the arithmetic, that decides the cost. Organising the codebook feature-major addresses exactly this, turning the search into a coalesced, tiled product in which each loaded weight column is reused across the samples of a tile - worth 4.5-8.5× on the search when nothing else is varied (5.1). Because the best-matching unit is an exact argmin, none of this touches quality; the speed-up is, in the strict sense, free.

Table 10: Whole-BMU-phase profiling.

Model	Edge	BMU (s/ep)	DRAM/ep (TB)	score block	DRAM BW (% of 1,008 GB/s)
SparseBin.SOM	128	9.63	2.82	- (fused)	29.0
cuSPARSE.SOM (feature-major)	128	8.72	3.10	1.74 (56%)	35.3
cuSPARSE.SOM (node-major)	128	65.64	3.84	1.73 (45%)	5.8
SparseBin.SOM	256	41.61	18.97	- (fused)	45.2
cuSPARSE.SOM (feature-major)	256	63.80	13.85	6.92 (50%)	21.5
cuSPARSE.SOM (node-major)	256	289.30	22.86	7.02 (31%)	7.8

Measured with Nsight Compute, full training split, RTX 4090, FP16 throughout. For cuSPARSE the phase sums the norm, sparse-dense product, row-pointer setup and argmin-reduction kernels; for SparseBin.SOM it is the single fused kernel plus a negligible norm pass. "Score block" is the dense score matrix the cuSPARSE path writes and reads back, which SparseBin.SOM never materialises. Sustained bandwidth here is phase DRAM divided by measured BMU time, expressed against the card's theoretical peak of 1,008 GB/s. Note that this is a different denominator from the per-kernel figures in Table 11, which are Nsight Compute's own dram___throughput percentages against its measured sustained peak of ≈982 GB/s (97.4% of theoretical); the two bases differ by 2.6% and are not interchangeable. The 128×128 captures are exact; at 256×256 cuSPARSE.SOM (feature-major) is sampled at 1-in-22 launches and cuSPARSE.SOM (node-major) at 1-in-329, validated to within ∼7% against the exact captures.

Table 11: Per-kernel decomposition of the cuSPARSE BMU phase.

Kernel	DRAM (TB)	Share of traffic	Time (s)	Share of time	DRAM BW (% of ≈982 GB/s)
Sparse-dense product (writes score block)	10.27	74%	15.44	25%	67.7
Argmin reduction (reads score block back)	3.57	26%	47.33	75%	7.7
Norm + row-pointer setup	<0.01	<1%	0.02	<1%	-

The mechanism in one table; all rows are the cuSPARSE feature-major path at 256×256. Percentages here are Nsight Compute's per-kernel dram___throughput against its measured sustained peak of ≈982 GB/s - a different denominator from Table 10, which uses the theoretical 1,008 GB/s; the two differ by 2.6%. On that basis the sparse-dense product streams at 67.7%; the argmin reduction that reads its score block back streams at 7.7%, so a quarter of the traffic consumes three-quarters of the time. The fused kernel performs both roles in one pass at 45.6% and never materialises the score block.

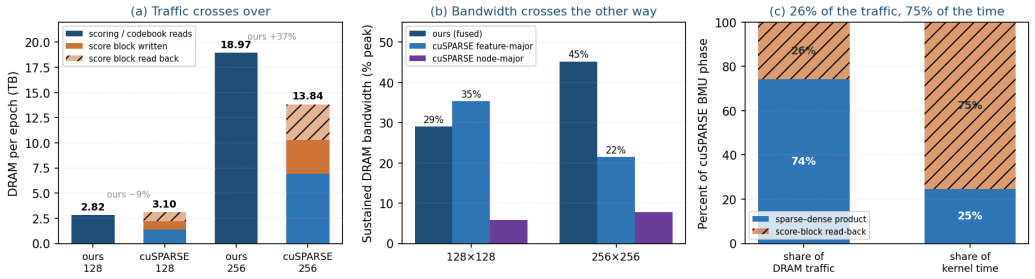

Figure 6: Whole-phase BMU profiling, FP16-matched. (a) DRAM per epoch, with the cuSPARSE bars split to show the write-then-read-back of the dense score block the fused kernel never materialises: the present traffic is 9% lower at 128×128 and 37% higher at 256×256. (b) Sustained DRAM bandwidth, crossing over the other way. (c) The mechanism: within the cuSPARSE phase at 256×256, the score-block read-back is a quarter of the traffic but three-quarters of the elapsed kernel time.

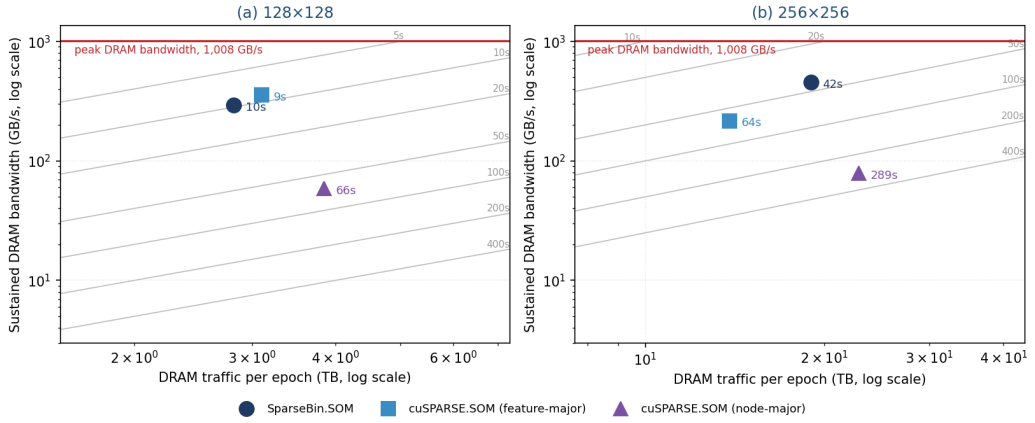

Figure 7: DRAM traffic against sustained bandwidth for the whole BMU phase, at 128×128 (a) and 256×256 (b). The horizontal line is the theoretical peak of 1,008 GB/s; the grey diagonals are contours of equal elapsed time, and each marker is annotated with the measured phase time. Moving up is streaming faster; moving right is moving more bytes; moving toward the upper left is finishing sooner. At 128×128 the three implementations separate almost entirely by bandwidth. At 256×256 the fused kernel sits to the right of the cuSPARSE feature-major path, moving 37% more data, yet well above it and on a faster iso-time contour, which is the crossover of 5.7 in one view.

Quality is instead governed by the update, which I keep inexpensive with a radius-independent box blur.

What the measurements then revise is why the complete design wins at large maps. The intuitive account - that a compact binary representation simply moves fewer bytes - does not survive precision-matching. At 256×256 SparseBin.SOM moves 18.97 TB of DRAM per epoch against the cuSPARSE baseline's 13.85 TB, some 37% more, and is nonetheless faster: 41.6 seconds against 63.8. Nor is the difference in the gaps between kernels, which whole-phase profiling bounds at a few per cent. It lies in what kind of traffic each design is obliged to move.

A sparse-dense product must materialise the dense score block in memory and read it back to find each sample's best unit, and that round trip alone costs 6.92 TB - half of everything that path moves - and, because the read-back is poorly localised, three-quarters of the elapsed kernel time. A fused search accumulates in registers and takes its two best units in the same sweep, so it pays nothing for that class of traffic at all, and streams the traffic it does move at more than twice the sustained bandwidth (45.2% against 21.5% of peak). The advantage is therefore the elimination of a category of memory traffic, not a reduction in the total; the absolute volume here is higher, and saying so is what makes the mechanism unambiguous.

Read this way the two design choices of Section 3 attack the same bottleneck by different means. Feature-major storage does not reduce the bytes the codebook read requires: it lets a warp fetch them in a few wide, cache-line-aligned transactions instead of many narrow ones, so the same traffic is delivered at a far higher fraction of peak bandwidth. Fusing the score and the argmin into a single pass does reduce the bytes, because the scores never reach memory at all and a whole class of traffic is never generated. One choice improves how efficiently data moves; the other removes data that would otherwise have to move. It is the combination - how the weights are laid out, and what is never written down - rather than any single clever kernel, that makes a full-corpus MEDLINE atlas tractable on commodity hardware. That combination carries it past a quarter of a million neurons on one consumer card, and beyond a million on a single datacentre GPU.

The approach is not specific to binary data. Replacing the gather-sum of the binary inner product with a gather-multiply - carrying $\|\mathrm{x}\|^2$ and a values array alongside the feature indices - would extend the same feature-major, fused-top-two, factorised-update design to sparse real-valued inputs at little additional cost in the large-map regime, where the shared O(M)

codebook read already dominates. The binary specialisation is most valuable at small and mid map sizes; a sparse-float variant would trade a little of that advantage for considerably broader applicability, which I regard as a worthwhile exchange and a natural next step.

A more conceptual implication concerns how the size of such a map ought to be chosen. The absence of an elbow in quantisation error is easily mistaken for an embarrassment - a failure to find the "right" size - when it is in fact a substantive finding. On a corpus whose topical structure is heavy-tailed and self-similar rather than concentrated at one granularity [7], there is no characteristic granularity for a flat grid to discover, so representational error simply improves with resolution until the compute budget is exhausted. The size question becomes meaningful only once a downstream objective is specified, and here the two objectives pull apart. Faithful quantisation rewards arbitrarily large maps; faithful recovery of the coarse knowledge structure, which I examine in the companion paper [5], saturates on quite small maps, because a flat lattice cannot embed an exponentially branching hierarchy beyond a shallow depth.

A concrete case makes the dissociation tangible. With roughly 29.9 million articles, a 50×50 map holds on average some twelve thousand papers per neuron, a 512×512 map about a hundred, and the 1,048,576-neuron map reported here roughly twenty-eight.

Consider the literature on CAR-T cell immunotherapy: even a 50×50 map places it correctly beside the rest of cancer immunotherapy and far from, say, cardiology, which is entirely adequate for browsing. At that resolution, however, the whole of it - B-cell-lymphoma trials, myeloma trials, bispecific T-cell engagers, the management of cytokine-release syndrome - collapses onto one or two neurons whose prototype is an average over twelve thousand papers, and these distinct research fronts become indistinguishable. Enlarging the map gives each front its own contiguous patch of neurons, so that near-duplicate detection, fine-grained nearest-neighbour retrieval, and the emergence of a genuinely new front on a previously empty neuron all become possible at the granularity of the front rather than the subfield. Recovering that the CAR-T region sits where it belongs is a small-map property; resolving within it is a large-map one, and it is this within-region resolution that quantisation error keeps rewarding without an elbow.

Finally, it is worth returning to the purpose that motivates the work. The value of a self-organising map of the medical literature was never its quantisation error in the abstract, but its promise as an objective, browsable

atlas of the kind that science mapping has long sought [6]. Against such an atlas, expert judgement about what to teach, to fund, and to research can be checked and, where necessary, corrected. That promise has been constrained less by theory than by the practical difficulty of building such a map over the whole corpus, at a resolution fine enough to be useful. The method described here removes much of that constraint: a full-MEDLINE atlas is now a matter of minutes on a single GPU, and its resolution can be set by the available compute rather than by a data-imposed ceiling. Whether the resulting atlas faithfully encodes the biomedical knowledge structure - against the MeSH tree and against independent, citation-derived taxonomies - is the question taken up in the companion validity paper [5].

7. Limitations

Several limitations qualify these results. All measurements were made on a single consumer architecture, the RTX 4090; the layout argument should hold on datacentre GPUs but I have not confirmed it there, and cross-GPU BMU identity is argued from the exactness of the argmin rather than demonstrated. The per-epoch advantage over cuSPARSE appears only above roughly 128×128 - below that the baseline is faster, and the wall-clock advantage at 128×128 rests partly on converging in fewer epochs rather than on raw speed. The DRAM comparison crosses over between the two profiled map sizes, so neither implementation can be described as uniformly leaner, and the 256×256 traffic figures are sampled and scaled rather than captured exhaustively (validated to within ∼7% against the exact 128×128 capture). Profiling covers two map sizes on one architecture; whether the same crossover appears at other sizes or on datacentre GPUs is untested. The somoclu comparison rests on a single map size, chosen as the most favourable point for it. The MedSOM comparison is timing-only, since its per-epoch codebook normalisation makes its quantisation error non-comparable. The size sweep is a single seed per rung, which is exact rather than uncertain because PCA-initialised full-batch training is deterministic, but it means the sweep carries no interval. Finally, kernel-shape, GPU-versus-CPU, and initialisation asymmetries between implementations are disclosed rather than eliminated.

8. Data and code availability

The anonymised corpus is available from Zenodo at `https://doi.org/10.5281/zenodo.20770707` under a CC0 1.0 public-domain dedication. This is the permanent version of the DOI that resolves to the current version of the record. The corpus is distributed unsplit, in the sparse-binary CSR format described in 4.2, and contains only binary MeSH descriptor vectors: no PubMed identifiers, no titles, no abstracts and no author or journal data. The 90/10 partition used throughout 5 is regenerated deterministically from it by the reproduction pipeline at seed 42, which also verifies the resulting file hashes against those recorded in the frozen manifest. The Zenodo deposit carries the corpus only. The frozen result files behind every table in 5 are released in that repository under `frozen/2026-08-02/`, together with the SHA-256 manifest that hashes them, so each reported figure can be traced to the file that produced it. Appendix Appendix A gives the mapping.

The corpus is derived from the PubMed 2026 annual baseline, courtesy of the U.S. National Library of Medicine. It reflects that baseline and does not reflect the most current data available from NLM; the U.S. National Library of Medicine has not endorsed this work, and any errors of derivation are the author's.

The implementation, the cuSPARSE baseline, the shared evaluator and the reproduction pipeline are released under the MIT licence at `https://github.com/mongrolwarrior/sparsesom-paper1` at tag `v1.0`, with pinned submodule commits recorded in the run provenance.

The pipeline provides two reproduction depths. The `outline` profile runs a single seed at reduced epoch budgets on the cheapest map size at which each effect is visible, and takes about three hours on one 24 GB GPU; it checks the direction and order of magnitude of every reported effect and reproduces no published value, range, interval or test. The `full` profile runs the complete matrix - five seeds, edges 32 to 512, every implementation - and takes approximately 6.5 days on the same card, after which the verification step scores the published claims; it may be run piecewise as four non-overlapping sections rather than in one pass. Nothing short of the full profile reproduces a statistic. Roofline analyses in the conventional arithmetic-intensity form are not included here, because the profiling captures behind 5.7 recorded memory traffic and timing but not floating-point counts; they will be provided as an addendum to the archived record.

9. Declarations

Author contributions. A.J.A. is the sole author and is responsible for the conception and design of the work, the implementation, the experiments, the analysis, and the writing.

Acknowledgements. The MEDLINE self-organising map programme on which this work builds was developed with Kyungmi Lee, Tarun Sen Gupta and Bunmi S. Malau-Aduli, whose contributions to the earlier studies cited here [1, 2, 3, 4] shaped the questions this paper addresses. The large-map experiments used a rented NVIDIA H200 instance. This research is indexed against data courtesy of the U.S. National Library of Medicine.

Funding. This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Ethics. This study analysed publicly available bibliographic metadata (MeSH descriptor annotations from the PubMed 2026 baseline). It involved no human participants, no animal subjects and no identifiable personal data, and required no ethics approval.

Declaration of generative AI in the manuscript preparation process. During the preparation of this work the author used generative AI assistance (Anthropic Claude) to draft and revise manuscript text, to structure and restructure sections, and to verify and format reference metadata. After using these tools the author reviewed and edited the content as needed and takes full responsibility for the content of the article. Use of the same assistance within the research process itself is described in 4.1. No AI system is credited as an author, and none is accountable for the work.

Appendix A. Provenance of tables and figures

Every table and figure in this paper is computed from a file in the frozen result set described in Section 8. The mapping is given below so that any quantity can be traced to the file it came from and recomputed by the scripts in the release.

Figures 1 and 2 are schematics and carry no measured data. Two further files in the same set support claims made in prose rather than in a table: `sigma0_sweep.parquet` for the σ_0 and initialisation comparison of 3.4, and `bringup_gate.json` for the bit-exactness check of 5.1.

Item	Content	Source file
Table 2	Corpus statistics	`corpus_manifest.json`
Table 3	Codebook layout in isolation	`impl_compare.parquet`
Table 4	Equal-quality crossover	`impl_compare.parquet`
Table 5	Equal-quality comparison	`impl_compare.parquet`, `efficiency_sweep.parquet`
Table 6	Effect of the update rule	`impl_compare.parquet`
Table 7	Neighbourhood-update cost	`impl_compare.parquet`
Table 8	Per-epoch cost at matched work	`efficiency_sweep.parquet`; somoclu clean re-measurement
Table 9	Held-out QE size sweep	`size_sweep.parquet`; H200 run log
Table 10	Whole-BMU-phase profiling	`roofline_phase_summary.csv`
Table 11	Per-kernel decomposition	`roofline_phase_breakdown.csv`
Figure 3	BMU share of training time	`size_sweep_epochs.parquet`
Figure 4	Efficiency chain	`efficiency_sweep.parquet`
Figure 5	Map-size sweep	`size_sweep.parquet`
Figure 6	Roofline profiling	`roofline_phase_summary.csv`, `roofline_phase_breakdown.csv`

References

[1] Amos, A. J., Lee, K., Sen Gupta, T. & Malau-Aduli, B. S. (2021). Systematic review of specialist selection methods with implications for diversity in the medical workforce. BMC Medical Education, 21, 448. doi:10.1186/s12909-021-02685-w

[2] Amos, A. J., Lee, K., Sen Gupta, T. & Malau-Aduli, B. S. (2022). Identifying emerging topics in the peer-reviewed literature to facilitate curriculum renewal and development. Current Psychology, 42, 30813-30824. doi:10.1007/s12144-022-04090-y

[3] Amos, A. J., Lee, K., Sen Gupta, T. & Malau-Aduli, B. S. (2024a). Defining the boundaries of psychiatric and medical knowledge: applying cartographic principles to self-organising maps. In MEDINFO 2023 - The Future Is Accessible, Studies in Health Technology and Informatics, vol. 310, pp. 795-799. Amsterdam: IOS Press. doi:10.3233/SHTI231074

[4] Amos, A. J., Lee, K., Sen Gupta, T. & Malau-Aduli, B. S. (2024b).

Validating the knowledge represented by a self-organizing map with an expert-derived knowledge structure. BMC Medical Education, 24, 416. doi:10.1186/s12909-024-05352-y

[5] Amos, A. J. (in preparation). What a self-organising map of MEDLINE encodes: convergent validity against the MeSH hierarchy and an independent citation-derived taxonomy.

[6] Börner, K. (2010). Atlas of Science: Visualizing What We Know. Cambridge, MA: MIT Press.

[7] Clauset, A., Shalizi, C. R. & Newman, M. E. J. (2009). Power-law distributions in empirical data. SIAM Review, 51(4), 661-703. doi:10.1137/070710111

[8] Crow, F. C. (1984). Summed-area tables for texture mapping. Computer Graphics (Proceedings of SIGGRAPH '84), 18(3), 207-212. doi:10.1145/964965.808600

[9] Kaski, S. & Lagus, K. (1996). Comparing self-organizing maps. In Artificial Neural Networks - ICANN'96, Lecture Notes in Computer Science 1112, pp. 809-814. Berlin: Springer. doi:10.1007/3-540-61510-5_136

[10] Kohonen, T. (2001). Self-Organizing Maps, 3rd edn. Springer Series in Information Sciences, vol. 30. Berlin: Springer. doi:10.1007/978-3-642-56927-2

[11] Kohonen, T. (2013). Essentials of the self-organizing map. Neural Networks, 37, 52-65. doi:10.1016/j.neunet.2012.09.018

[12] Kohonen, T., Kaski, S., Lagus, K., Salojarvi, J., Honkela, J., Paatero, V. & Saarela, A. (2000). Self-organization of a massive document collection. IEEE Transactions on Neural Networks, 11(3), 574-585. doi:10.1109/72.846729

[13] Lipscomb, C. E. (2000). Medical Subject Headings (MeSH). Bulletin of the Medical Library Association, 88(3), 265-266.

[14] Motegi, R. & Seki, Y. (2023). SMLSOM: the shrinking maximum likelihood self-organizing map. Computational Statistics & Data Analysis, 182, 107714. doi:10.1016/j.csda.2023.107714

[15] Muggeo, V. M. R. (2003). Estimating regression models with unknown break-points. Statistics in Medicine, 22(19), 3055-3071. doi:10.1002/sim.1545

[16] NVIDIA Corporation (2025). cuSPARSE Library, CUDA Toolkit 12.8. https://docs.nvidia.com/cuda/cusparse/ [accessed 2026-07-30].

[17] NVIDIA Corporation (2025). Nsight Compute, CUDA Toolkit 12.8. https://docs.nvidia.com/nsight-compute/ [accessed 2026-07-30].

[18] Pramanik, A., Sarkar, S., Maiti, J. & Mitra, P. (2021). RT-GSOM: Rough tolerance growing self-organizing map. Information Sciences, 566, 19-37. doi:10.1016/j.ins.2021.01.039

[19] Rauber, A., Merkl, D. & Dittenbach, M. (2002). The growing hierarchical self-organizing map: exploratory analysis of high-dimensional data. IEEE Transactions on Neural Networks, 13(6), 1331-1341. doi:10.1109/TNN.2002.804221

[20] Vesanto, J. & Alhoniemi, E. (2000). Clustering of the self-organizing map. IEEE Transactions on Neural Networks, 11(3), 586-600. doi:10.1109/72.846731

[21] Wells, W. M. (1986). Efficient synthesis of Gaussian filters by cascaded uniform filters. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(2), 234-239. doi:10.1109/TPAMI.1986.4767776

[22] Williams, S., Waterman, A. & Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65-76. doi:10.1145/1498765.1498785

[23] Wittek, P., Gao, S. C., Lim, I. S. & Zhao, L. (2017). somoclu: An efficient parallel library for self-organizing maps. Journal of Statistical Software, 78(9), 1-21. doi:10.18637/jss.v078.i09